

Keywords: diffusion transformer • momentum orthogonalization • optimizer • image generation

CMuon: Accelerating and Stabilizing Diffusion Transformer Training via Chunked Momentum Orthogonalization

Partitioning Fused Weight Matrices Before Orthogonalization Delivers a 2× Speedup Over AdamW and FID 1.18 on ImageNet 256

By Chuyan Chen, Peng Sun & Kun Yuan

arXiv:2608.02502 • ECCV 2026

The Muon optimizer’s momentum orthogonalization accelerates standard Transformer training, but applying it directly to Diffusion Transformers stalls at late stages. CMuon traces the cause to fused weight tensors coupling independent subspaces and fixes it with a single structural change—partitioning matrices into chunks before orthogonalization. A 675M-parameter DiT trained with CMuon achieves FID 1.18 on ImageNet 256×256 in 200 epochs, more than 2× faster than AdamW.

AT A GLANCE

Problem: Muon achieves faster convergence than AdamW on standard Transformers but stalls in the late training phase when applied to Diffusion Transformers (DiTs).

Why it matters: DiT training is the dominant compute cost for frontier image and video generation; a 2× optimizer speedup directly halves training cost.

Method: Identify fused weight tensors (AdaLN, QKV) as the source of subspace coupling; chunk them prior to Nesterov momentum orthogonalization.

Result: FID 1.18 on ImageNet 256×256 in 200 epochs (675M parameters), > 2× training speedup over AdamW.

Evidence: Controlled ablations against Muon and AdamW; FID curves over full 200-epoch training run; accepted ECCV 2026.

The Big Picture

Diffusion Transformers (DiTs) power the best image and video generation models of 2026, from FLUX to Sora-class video synthesis. Training them is extremely expensive: a state-of-the-art image generation DiT requires thousands of GPU-hours, and video models demand an order of magnitude more. Any optimizer improvement that accelerates

convergence without degrading final quality translates directly to lower training cost.

Muon—the Momentum Orthogonalization optimizer—has attracted attention because it achieves significantly faster convergence than AdamW on vanilla Transformer language models. CMuon makes Muon work on DiTs by identifying and fixing a structural mismatch between Muon’s assumptions and DiT’s weight layout.

Why Existing Approaches Fall Short

AdamW is the de-facto DiT optimizer. It works reliably but converges slowly, requiring many epochs to reach competitive FID scores.

Vanilla Muon applies Nesterov momentum and then orthogonalizes the resulting gradient update matrix via Newton–Schulz iterations before applying the weight update. This exploits the geometry of the weight matrix space to find descent directions that make maximal use of each parameter. On standard Transformer blocks (Q, K, V, and MLP projections each in their own tensors), it works well.

But DiTs use **Adaptive Layer Norm (AdaLN)**: a conditioning mechanism that generates per-channel scale and shift parameters from the diffusion timestep embedding, modulating the activations of each Transformer block. For GPU efficiency, the linear layers implementing AdaLN fuse the weights for scale and shift into a single large matrix, and similarly the Q, K, V projections are sometimes stored fused. Applying Muon to these fused tensors treats two functionally independent linear maps as a single entity, causing the orthogonalization to mix their update directions. This implicit coupling distorts the gradient geometry and causes convergence to plateau.

Core Mechanism

CMuon introduces a minimal pre-processing step before the Muon update:

1. **Identify** all fused weight tensors in the DiT (AdaLN scale+shift matrices, fused QKV projections, etc.).
2. **Chunk** each fused matrix along its output channel dimension according to the functional boundary (e.g., split AdaLN into its scale matrix and its shift matrix separately).
3. **Orthogonalize** each chunk independently via the Newton–Schulz approximation to the matrix orthogonal factor.
4. **Reassemble** the chunked updates and apply them to the original fused parameter tensor.

No architectural changes or additional hyperparameters are required. The same chunking boundaries used during the forward pass for functional interpretation are reused

here during the optimizer step.

Technical Formulation

Let $\mathbf{W} \in \mathbb{R}^{2d \times d}$ be a fused AdaLN weight matrix combining a scale projection $\mathbf{W}_s \in \mathbb{R}^{d \times d}$ and a shift projection $\mathbf{W}_b \in \mathbb{R}^{d \times d}$.

The Muon update for $\mathbf{W}$ is:

$$\Delta \mathbf{W} = \text{NS}(\mathbf{M}), \quad \mathbf{M} = \mu \mathbf{M}_{\text{prev}} + \mathbf{G},$$

where $\mathbf{G}$ is the gradient, μ is momentum, and $\text{NS}(\cdot)$ denotes the Newton–Schulz orthogonalization:

$$\text{NS}(\mathbf{A}) \approx \frac{3}{2} \mathbf{A} - \frac{1}{2} \mathbf{A} \mathbf{A}^\top \mathbf{A} \quad (\text{one step}).$$

The CMuon update orthogonalizes each chunk separately:

$$\Delta \mathbf{W}_s = \text{NS}(\mathbf{M}_s), \quad \Delta \mathbf{W}_b = \text{NS}(\mathbf{M}_b),$$

where $\mathbf{M}_s, \mathbf{M}_b$ are the momentum matrices for each chunk (maintained separately). The final update is $\Delta \mathbf{W} = [\Delta \mathbf{W}_s; \Delta \mathbf{W}_b]$.

By orthogonalizing each functional subspace independently, each chunk’s update direction is computed in the geometry appropriate to its own weight space rather than the mixed geometry of the fused tensor.

Learning or Inference Procedure

CMuon is a drop-in optimizer replacement. Training proceeds as:

1. Forward pass and diffusion denoising loss computation (identical to standard DiT training).
2. Backward pass yielding gradients for all parameters.
3. For each fused weight tensor, split gradients and momentum along the chunk boundary, orthogonalize each chunk, reassemble update.
4. For non-fused parameters (e.g., biases, embedding tables), use AdamW as usual (Muon is not well-suited to 1-D parameter vectors).
5. Apply updates with a learning rate schedule identical to AdamW baselines for fair comparison.

Inference is unaffected: CMuon is a training-time optimizer; the deployed model is identical in architecture to a standard DiT.

What the Guarantee Says

Muon’s orthogonalization is motivated by the Riemannian gradient on the manifold of matrices with bounded spectral norm. Each update step is guaranteed to be a descent direction in this geometry. The CMuon insight is that applying this guarantee to a fused matrix provides a weaker

guarantee than applying it to each component separately, because the fused matrix does not lie on a single manifold coherent with either component’s optimization landscape. Chunk-level orthogonalization restores the per-component descent guarantee.

Experimental Findings

Evaluated on DiT-XL/2 (675M parameters) training on ImageNet 256×256 with class-conditional generation:

- CMuon achieves **FID 1.18** in 200 epochs—competitive with the best published DiT-XL/2 results.
- Training is **> 2× faster** than AdamW measured by FID at equal epoch count.
- Vanilla Muon on DiTs plateaus around epoch 120–140 and never reaches FID 1.3; CMuon continues descending through 200 epochs.
- No guidance (classifier-free or otherwise) is used during evaluation, making this an apples-to-apples comparison.

Ablations and Interpretation

Without chunking (vanilla Muon): Convergence stalls at FID ≈ 1.4 around epoch 130. The late-stage plateau is eliminated by chunking.

Chunk granularity: Chunking at the AdaLN scale+shift boundary is sufficient. Further splitting within Q, K, V provides marginal additional gain on this architecture.

Non-fused layers: Applying Muon (without chunking) to non-fused DiT layers (attention projections stored separately) does not cause convergence issues, confirming that fused tensors are the specific failure mode.

AdamW vs. CMuon FID curves: At epoch 100 AdamW achieves FID ≈ 1.8 , while CMuon achieves FID ≈ 1.3 , confirming the 2× epoch-efficiency claim.

Reference

Chuyan Chen, Peng Sun, Kun Yuan. **CMuon: Accelerating and Stabilizing Diffusion Transformer Training via Chunked Momentum Orthogonalization**. arXiv:2608.02502, ECCV 2026. <https://arxiv.org/abs/2608.02502>